

Optional Section

This section is optional. Create a new directory (1ab67-section4/) and open it in Kiro IDE as a fresh workspace.

These are just ideas

The use cases below are starting points to spark your creativity - not step-by-step labs. Pick one that resonates with your team, or come up with your own idea entirely. Use the patterns you learned in Sections 2 and 3 to build it.

Choose Your Use Case

You've learned the core concepts (Section 2) and advanced patterns (Section 3). Now pick a use case and build it yourself.

Use Case	What to Build	Difficulty	Est. Time
SRE Assistant	Agent + 4 Skills	Hard	30-45 min
CIS Benchmark Validator	Agent + 1 Skill	Medium	20-30 min
Log Analyzer	1 Skill	Easy	10-15 min
Documentation Generator	Agent + 2 Skills	Medium	20-30 min
IaC Creator & Reviewer	Agent + Power	Medium	20-30 min
Your Own	Varies	Varies	Varies

Use Case 1: SRE Assistant

Custom Agent + Skills - An agent for day-to-day SRE operations: recommend, troubleshoot, create change requests, and post-incident reports.

Example: "Database response time increased 3x since last deployment" -> Prioritized checklist with specific commands to run.

Agent Config

IDE (.md)CLI (.json)

```
1 # .kiro/agents/sre-assistant.md
2 ---
3 name: sre-assistant
4 description: >
5   SRE assistant for incident response, troubleshooting,
6   change management, and post-incident reviews.
7   tools: ["read", "write", "shell", "@fetch"]
8   model: auto
9   ---
10
11 You are an expert SRE. Help with incident response, troubleshooting,
12 change management, and post-incident reviews.
13
14 ## How You Work
15 1. Use the sre-recommend skill for operational recommendations
16 2. Use the sre-pir skill for post-incident reports
17 3. Reference runbooks in the workspace when available
18 4. Always provide actionable steps with specific commands
```

Skills to Create

.kiro/skills/sre-recommend/SKILL.md

```
1 # .kiro/skills/sre-recommend/SKILL.md
2 ---
3 name: sre-recommend
4 description: >
5   Provide actionable SRE recommendations for operational situations.
6   Use when someone describes a production issue or asks for operational advice.
7   ---
8
9 # SRE Recommendation Framework
10 ## Analysis Steps
11 1. Identify the symptom category (latency, errors, saturation, traffic)
12 2. Map to potential root causes using USE method
13 3. Prioritize by blast radius and likelihood
14 4. Provide actionable steps with specific commands
15
16 ## Output Format
17 ### Situation Summary
18 ### Prioritized Actions
19 | Priority | Action | Command/Steps | Expected Outcome |
20 |-----|-----|-----|-----|
21 ### Escalation Criteria
```

.kiro/skills/sre-pir/SKILL.md

```
1 # .kiro/skills/sre-pir/SKILL.md
2 ---
3 name: sre-pir
4 description: >
5   Generate structured Post-Incident Reports from incident descriptions.
6   Use when someone describes a past incident and needs a PIR document.
7   ---
8
9 # Post-Incident Report Template
10 ## Sections
11 1. ** Incident Summary **: What happened, severity, duration
12 2. ** Timeline **: Chronological events with timestamps
13 3. ** Root Cause Analysis **: 5 Whys technique
14 4. ** Impact **: Users affected, SLA impact, financial impact
15 5. ** Resolution **: What fixed it and why
16 6. ** Action Items **: Preventive measures with owners and deadlines
17 7. ** Lessons Learned **: What went well, what didn't
```

Use Case 2: CIS Benchmark Validator (SecOps)

Custom Agent + Skill - Connect to a Linux server via SSH, run audit commands, validate against CIS benchmark controls.

Example: "Check server 10.0.1.50 against CIS Level 1 benchmarks" -> Table of CIS controls with pass/fail status and remediation.

Agent Config

IDE (.md)CLI (.json)

```
1 # .kiro/agents/cis-validator.md
2 ---
3 name: cis-validator
4 description: >
5   CIS Benchmark validator for Linux servers via SSH.
6   Use when auditing server security configuration.
7   tools: ["read", "write", "shell"]
8   model: auto
9   ---
10
11 You are a CIS Benchmark auditor. When given a server address:
12 1. Use SSH to connect and run audit commands
13 2. Compare results against CIS Level 1 controls
14 3. Report findings with CIS control IDs
15 4. Provide exact remediation commands
16
17 Always output a compliance summary table.
```

Skill to Create

.kiro/skills/cis-linux-l1/SKILL.md

```
1 # .kiro/skills/cis-linux-l1/SKILL.md
2 ---
3 name: cis-linux-l1
4 description: >
5   CIS Level 1 benchmark controls for Linux servers.
6   Use when auditing or validating Linux server security configuration.
7   ---
8
9 # CIS Level 1 Linux Benchmark Checks
10
11 ## 1. Filesystem Configuration
12 - 1.1.1 Ensure mounting of cramfs is disabled
13   Command: modprobe -n -v cramfs | grep -E '(cramfs|install)'
14   Expected: install /bin/true
15
16 ## 2. SSH Server Configuration
17 - 5.2.1 Ensure permissions on /etc/ssh/sshd_config
18   Command: stat /etc/ssh/sshd_config
19   Expected: Access: (0600/-rw-----)
20
21 ## 3. Audit Configuration
22 - 4.1.1 Ensure auditd is installed
23   Command: rpm -q audit-audit-libs || dpkg -s auditd
24   Expected: Package installed
25
26 ## Output Format
27 | CIS ID | Control | Status | Finding | Remediation |
28 |-----|-----|-----|-----|-----|
```

Use Case 3: Log Analyzer & Summarizer

Skill only - The simplest use case. Paste logs in any format and get structured analysis.

Example: 500 lines of syslog -> "Between 02:00-02:45, 47 failed SSH login attempts from 3 IPs. /var/log reached 92% at 03:15. Recommend: block IPs, rotate credentials, expand /var/log."

.kiro/skills/log-analyzer/SKILL.md

```
1 # .kiro/skills/log-analyzer/SKILL.md
2 ---
3 name: log-analyzer
4 description: >
5   Analyze and summarize log files of any format. Use when someone pastes
6   logs or asks to analyze syslog, application logs, firewall logs, or access logs.
7   ---
8
9 # Log Analysis Framework
10 ## Steps
11 1. ** Identify log format **: syslog, JSON, Apache/Nginx access, application-specific
12 2. ** Parse and categorize **: errors, warnings, auth events, system events
13 3. ** Detect patterns **: repeated failures, time-based clusters, source correlations
14 4. ** Build timeline **: group related events chronologically
15 5. ** Identify anomalies **: unusual patterns, spikes, new sources
16
17 ## Output Format
18 ### Summary (plain language, 2-3 sentences)
19 ### Timeline
20 | Time Range | Event Category | Count | Details |
21 |-----|-----|-----|-----|
22 ### Anomalies & Findings
23 ### Recommended Next Steps (prioritized)
```

Use Case 4: Documentation Generator

Custom Agent + Skill - Generate SOPs, architecture docs, handover docs from descriptions or configs.

Example: "3 web servers behind F5, MySQL primary + replica, nightly cron backups to NFS, Zabbix monitoring" -> System architecture document with component inventory, data flow, and procedures.

Agent Config

IDE (.md)CLI (.json)

```
1 # .kiro/agents/doc-generator.md
2 ---
3 name: doc-generator
4 description: >
5   Generate structured infrastructure documentation from
6   descriptions or configs. Supports SOPs, architecture docs,
7   handover docs, and config documentation.
8   tools: ["read", "write"]
9   model: auto
10   ---
11
12 You are a technical documentation specialist for infrastructure teams.
13 Generate clean, structured docs in the requested format.
14
15 ## Always Include
16 - Overview and purpose
17 - Component inventory with versions
18 - Data flow and dependencies
19 - Operational procedures (backup, monitoring, scaling)
20
21 ## Output
22 Markdown format suitable for team wikis.
```

Use Case 5: IaC Creator & Reviewer

Custom Agent + Power - Two modes: create or review IaC for any platform.

Power Reuse

If you completed Section 3, reuse the power-infra-review Power with `includePowers: true`. If you skipped Section 3, install the **Terraform** power from the [marketplace](#).

Agent Config

IDE (.md)CLI (.json)

```
1 # .kiro/agents/iaC-expert.md
2 ---
3 name: iaC-expert
4 description: >
5   Infrastructure-as-Code creator and reviewer for Terraform,
6   CloudFormation, and Ansible.
7   tools: ["read", "write", "shell"]
8   model: auto
9   includePowers: true
10   ---
11
12 You are an IaC expert. Two modes:
13
14 ## Create Mode
15 Generate production-ready IaC with proper module structure,
16 security best practices, required tags, and outputs.
17
18 ## Review Mode
19 Review for security misconfigurations, hardcoded values,
20 missing tags, drift risks. Provide corrected code snippets.
```

Use Case 6: Bring Your Own

Have a use case specific to your team? Start here:

1. Identify a task you repeat often (reviews, audits, reports, onboarding)
2. Decide the scope: Skill (workflow only), Power (workflow + MCP tools), or Agent (dedicated role)
3. Start with a Steering file for project context, then layer on a Skill or Power

Starter Templates

SkillAgent (IDE)Agent (CLI)

```
1 # .kiro/skills/my-skill/SKILL.md
2 ---
3 name: my-skill
4 description: >
5   Describe what this skill does and when to activate it.
6   ---
7
8 ## Step 1: Gather Context
9 ## Step 2: Process
10 ## Step 3: Output
```

Share with Your Team

Commit your `.kiro/` directory to your repository. Steering files, skills, and agents in workspace scope are automatically available to all team members who open the project.